

35.3. La fonction `cachier16()` — ôter un personnage de 16×16

`#include <cachier16>` : `cachier16(numero)` ôte les quatre lutins d'un personnage de 16×16 .

1. Ce qu'il faut comprendre

Ce qu'elle fait : `cachier16(numero)` ôte les quatre lutins `numero` à `numero + 3` : tout le personnage.

La ligne à écrire : `#include <cachier16>`.

Son argument : le numéro du premier lutin, comme pour `sprite16()`.

Ce qu'elle coûte : environ **105 octets** avec la mise en place des lutins.

Essaie : fais-le réapparaître à l'image 240.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **356 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	47 o	← la console, à l'allumage – la mise en route de la console
main()	126 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
Tuiles	64 o	← CopierTuiles – 0 lettre sur 43, et 4 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert – la petite routine qui copie les lutins

Lettres gravées : aucune

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `EffacerCarte`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

<code>sprite()</code>	x 4	96 o sur place, dans main()
<code>cachier16()</code>	x 1	13 o sur place, dans main()
<code>image()</code>	x 1	6 o sur place, dans main() + AttendreImage (28 o)
<code>images()</code>	x 1	2 o sur place, dans main()
<code>sprite16()</code>	x 1	0 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <cachier16> : ôter un personnage de 16 x 16 ----
// BONHOMME est dessiné dans perso_BONHOMME.cpp ; ici, on ne fait que l'appeler.

#include <sprite16>           // sprite16() : le poser
#include <cachier16>          // cachier16() : ôter ses quatre lutins
#include "personnages.cpp"    // la liste : BONHOMME
```

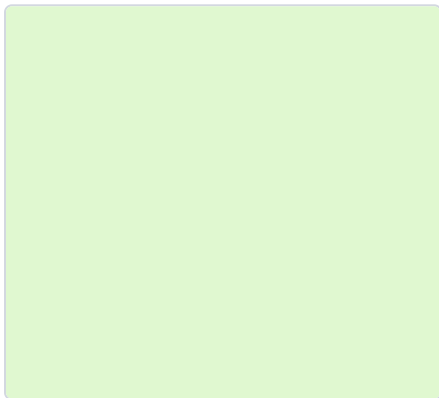
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    sprite16(0, 72, 64, BONHOMME);    // les lutins 0, 1, 2 et 3

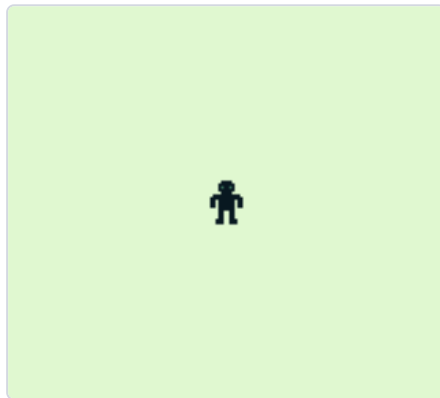
    while (true) {
        image();
        if (images() == 120) {
            cacher16(0);                // les lutins 0, 1, 2 et 3 s'en vont
        }
    }
}
```

3. Ce qui se passe, étape par étape

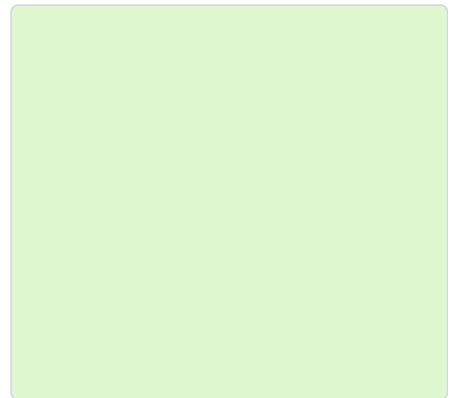
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le bonhomme au milieu, qui disparaît au bout de deux secondes.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, BONHOMME est là
- ✓ puis ses quatre lutins sont hors de l'écran

Ensuite : **35.4. La fonction `sprite32()` — un grand personnage de 32 × 32** — chapitre 6 : Dessiner. · [gameboy3](#) — écrire une cartouche Game Boy en C++